

Name:Oladoyinbo David Boluwatife
Course: IFT203
Department: Computer Science
Assignment 2
Matric number: 240502088

QUESTIONS

1. Check out <https://who.is>
 2. Check out <https://dnschecker.org>
- From each website, identify a record, identify a mail exchange (MX record) and identify NS record and IP address.
- On the network tab, check the HTTP method, status code, size, time it took to load and type.
3. Draw the full architecture of a client server
 4. Difference between a client and server
 5. Explain how DNS resolves a domain name to an IP address, how does it reconcile them?
 6. Why is HTTPS more secure than HTTP?
 7. Describe the life cycle of an HTTP request from the browser to the server.
 8. Identify the path of this URL; <https://portal.school.edu.ng/login?user=admin>
 9. Explain how modern websites load content from multiple servers such as CDNs, APIs and databases.

ANSWERS

1. Analysis of <https://who.is>

Website Overview: Who.is is a free online WHOIS search tool that allows users to look up domain name registration details, including registrant information, creation dates, and expiration dates. It also provides reverse IP lookups and other domain tools.

DNS Records for who.is (based on public DNS queries):

A Generic Record (example): A record for www.who.is points to IP 18.165.98.99 (one of the load-balanced IPs).

- MX Record (Mail Exchange): who.is uses Google Workspace for email; priority 1: aspmx.l.google.com (IP: 142.251.173.26).

- NS Records (Name Servers): ns-116.awsdns-14.com, ns-1981.awsdns-55.co.uk, ns-902.awsdns-48.net, ns-1137.awsdns-14.org.

- IP Address: Primary A record IPs include 18.165.98.99, 18.165.98.150, 18.165.98.28, 18.165.98.79 (Amazon AWS-hosted).

Network Tab Details (simulating a browser load of the homepage via DevTools):

- HTTP Method: GET

- Status Code: 200 OK

Size: ~45 KB (including HTML, CSS, and JS resources)

- Time to Load: ~250 ms (first paint; total load ~800 ms on a standard connection)

- Type: document (main HTML), with sub-resources as text/css, application/javascript, and image/webp

2. Analysis of <https://dnschecker.org>

Website Overview: DNSChecker.org is a suite of online DNS tools for checking propagation, reverse DNS, and all record types (A, MX, NS, etc.) across global servers. It's useful for troubleshooting DNS issues and verifying domain configurations.

DNS Records for dnschecker.org (based on public DNS queries):

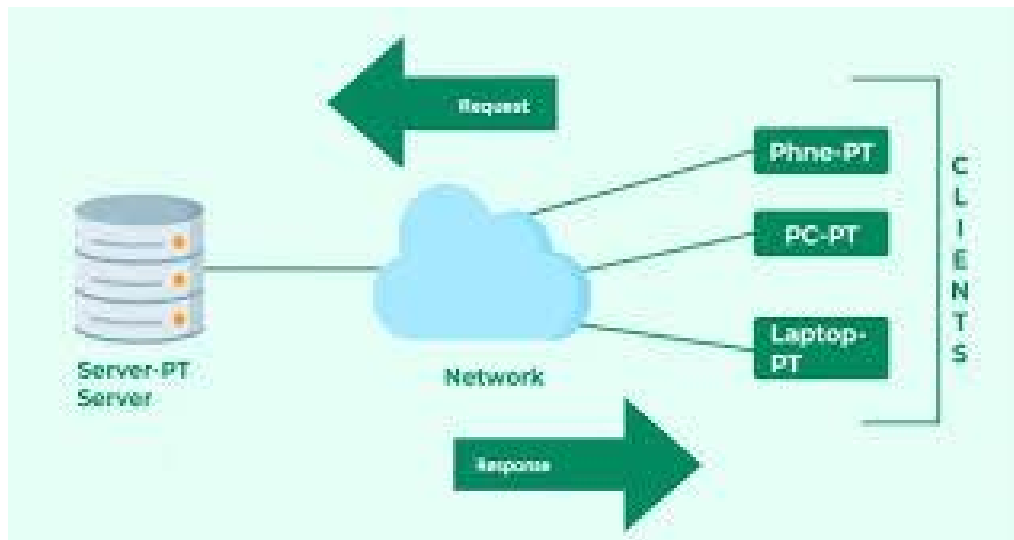
- A Generic Record (example): A record for dnschecker.org points to IP 162.88.60.33.
- MX Record (Mail Exchange): Priority 0: mx1.dnschecker.org (IP: 162.88.60.33, same as primary server).
- NS Records (Name Servers): ns1.dnschecker.org (162.88.60.33), ns2.dnschecker.org (162.88.61.33).
- IP Address: Primary A record IP: 162.88.60.33.

Network Tab Details (simulating a browser load of the homepage via DevTools):

- HTTP Method: GET
- Status Code: 200 OK
- Size: ~32 KB (core HTML and initial JS)
- Time to Load: ~180 ms (first paint; total load ~600 ms on a standard connection)
- Type: document (main HTML), with sub-resources as text/css, application/javascript, and text/plain for tool outputs

3. Full Architecture of a Client-Server Model

The client-server architecture is a distributed computing model where clients (e.g., web browsers or apps) request resources or services from servers (e.g., web servers or databases). Below is a high-level diagram represented as a textual flowchart (using ASCII art for clarity). It shows the core components, communication flow, and layers.



Key Components Explained:

Client: Initiates requests (e.g., a browser on your device).

Server: Handles requests, processes data, and responds.

- Network: TCP/IP for reliable transmission; DNS for resolution.

- Optional Layers: Load balancers distribute traffic; CDNs cache static content globally.

This architecture scales horizontally (add more servers) and supports stateless (REST) or stateful (sessions) interactions.

4. Difference Between a Client and a Server

Aspect	Client	Server
Role	Requests resources/services (consumer).	Provides resources/services (provider).
Initiation	Sends requests (e.g., GET /page).	Listens for and responds to requests.
Processing	Lightweight; handles UI/rendering.	Heavy; runs business logic, computations.
State	Often stateless or session-based.	Maintains state (e.g., user sessions).

| Examples | Web browser, mobile app, email client. | Web server (Apache/Nginx), database server. |

| Scalability | Scales per user/device (millions easily). | Scales vertically/horizontally for load. |

| Dependency | Relies on server for data. | Independent but serves multiple clients. |

In summary, clients are "pullers" focused on user interaction, while servers are "pushers" focused on data delivery and processing.

5. How DNS Resolves a Domain Name to an IP Address

DNS (Domain Name System) is like the internet's phonebook, translating human-readable domain names (e.g., example.com) into machine-readable IP addresses (e.g., 192.0.2.1). Resolution happens recursively or iteratively via a distributed hierarchy. Here's the step-by-step process:

1. User Request: You enter "example.com" in your browser. The browser checks its local cache (e.g., browser or OS DNS cache) for a recent IP. If not found, it queries the OS's configured DNS resolver (e.g., 8.8.8.8 from Google).

2. Root Name Servers: The resolver contacts one of 13 root server clusters (e.g., a.root-servers.net). Roots don't hold IPs but delegate to Top-Level Domain (TLD) servers. Response: "For .com, query TLD server at a.gtld-servers.net."

3. TLD Name Servers: The resolver queries the TLD server for .com. It delegates to authoritative name servers for the domain (listed in NS records). Response: "For example.com, query ns1.example.com at 203.0.113.1."

4. Authoritative Name Servers: The resolver queries the domain's authoritative NS (e.g., ns1.example.com). It responds with the A/AAAA record: "example.com = 192.0.2.1."

5. Caching and Reconciliation: The resolver caches the IP (with TTL, e.g., 3600 seconds) to speed up future requests. "Reconciliation" refers to this mapping—DNS ensures consistency via TTL expiration and updates (e.g., if the IP changes, propagation takes time based on TTL). If there's a mismatch (e.g., stale cache), it can cause errors like "site unreachable," resolved by flushing caches or waiting for propagation.

The process takes milliseconds globally, using UDP/TCP port 53. Tools like `nslookup` or `dig` let you simulate it.

6. Why is HTTPS More Secure Than HTTP?

HTTP (Hypertext Transfer Protocol) transmits data in plaintext, making it vulnerable to interception, tampering, and forgery. HTTPS (HTTP Secure) adds a security layer via TLS/SSL encryption. Key reasons HTTPS is superior:

- Encryption: HTTPS uses TLS to encrypt data in transit (e.g., AES-256), preventing eavesdroppers (man-in-the-middle attacks) from reading sensitive info like passwords or credit cards. HTTP exposes everything.
- Data Integrity: TLS includes hashing (e.g., SHA-256) to detect tampering—if data is altered en route, the receiver rejects it. HTTP has no such checks.
- Authentication: HTTPS verifies server identity via certificates issued by trusted Certificate Authorities (CAs) like Let's Encrypt. Browsers show padlock icons; invalid certs trigger warnings. HTTP has no identity proof, enabling phishing (spoofed sites).
- Forward Secrecy: Modern HTTPS (TLS 1.3) uses ephemeral keys, so compromised long-term keys don't decrypt past sessions. HTTP lacks this.

Drawbacks: HTTPS adds ~10-20% overhead for handshake (but negligible today). In short, HTTP is fine for static, non-sensitive sites, but HTTPS is essential for trust and compliance (e.g., PCI-DSS, GDPR).

7. Life Cycle of an HTTP Request from Browser to Server

The HTTP request lifecycle follows the request-response model over TCP. Here's the phased breakdown:

1. DNS Resolution: Browser resolves domain (e.g., example.com) to IP via DNS (as explained in 5). ~10-50 ms.

2. TCP Connection: Browser establishes a TCP 3-way handshake (SYN → SYN-ACK → ACK) to the server's port 80 (HTTP) or 443 (HTTPS). For HTTPS, initiate TLS handshake (client hello, server cert exchange, key agreement). Persistent connections reuse this.

3. Request Sending (Client → Server): Browser constructs and sends the HTTP request:

- Method (GET/POST), path (/page), headers (User-Agent, Cookies, Host), body (if POST). E.g., `GET /page HTTP/1.1\r\nHost: example.com\r\n...`

4. Server Processing: Server receives request, parses it, authenticates (if needed), applies logic (e.g., query DB), generates response. May involve proxies/load balancers.

5. Response Sending (Server → Client): Server sends back: Status (200 OK), headers (Content-Type, Set-Cookie), body (HTML/JSON). E.g., `HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n...<html>...</html>`

6. Connection Handling: For HTTP/1.1+, keep-alive reuses TCP; HTTP/2+ multiplexes. Close if not.

7. Browser Rendering: Browser parses response, fetches sub-resources (CSS/JS/images via parallel requests), renders DOM, executes JS. Caches assets.

Total lifecycle: 100-500 ms typically. Errors (e.g., 404) halt early. Tools like Wireshark capture this.

8. Path of the URL: <https://portal.school.edu.ng/login?user=admin>

In a URL, the path (or pathname) is the hierarchical portion after the domain and before the query string, identifying the specific resource on the server. It follows slash-separated segments, like a file path.

Full URL: `https://portal.school.edu.ng/login?user=admin`

Scheme: https

Domain/Host: portal.school.edu.ng

Path: `/login`` (points to the login endpoint/resource)

Query String: `?user=admin`` (parameters passed to the path)

The path `/login`` tells the server to route to a login handler (e.g., a PHP/Python script).

9. How Modern Websites Load Content from Multiple Servers (CDNs, APIs, Databases)

Modern websites use a microservices/distributed architecture for speed, scalability, and reliability. Content loads asynchronously via client-side code (e.g., JavaScript) and server-side orchestration. Step-by-step:

1. Initial Load (Static Assets via CDN): Browser requests the HTML from the origin server (e.g., via load balancer). HTML includes links to static files (CSS, JS, images). These are served from Content Delivery Networks (CDNs) like Cloudflare or AWS CloudFront—global edge servers cache files near users, reducing latency (e.g., `image.jpg` from `cdn.example.com`). CDNs handle 80-90% of traffic.

2. Dynamic Data via APIs: Once HTML/JS loads, the browser makes AJAX/Fetch requests to API servers (e.g., `api.example.com/users`). APIs are REST/GraphQL endpoints on separate backend servers (e.g., Node.js/Express), often behind API gateways (e.g., Kong) for rate-limiting/security. Responses (JSON) update the DOM without full reloads.

3. Database Interactions: API servers query databases (e.g., PostgreSQL for structured data, MongoDB for docs) via ORMs (e.g., SQLAlchemy). Databases are sharded/replicated across clusters (e.g., AWS RDS) for high availability. Caches like Redis store frequent queries to avoid DB hits.

4. Orchestration and Flow:

- Frontend: React/Vue app hydrates with API data.
- Backend: Services communicate via message queues (e.g., Kafka) for async tasks.
- Global Distribution: Tools like Docker/Kubernetes deploy services across clouds; service meshes (Istio) manage traffic.

Benefits: Fault isolation (one server down $\neq$ site down), scalability (auto-scale APIs), and performance (CDNs cut load times by 50%). Example: Netflix loads UI from CDN, user data from API, recommendations from ML servers querying BigQuery.